BUGTRIAGE AI

Clasificación asistida de reportes de errores mediante un modelo de IA preentrenado

Trabajo práctico integrador

Seminario de Actualización 2026 · IFTS N.º 18

Autora: Gabriela Coronel

Aplicación publicada: Hugging Face Spaces — BugTriage AI

Objetivo: construir, evaluar y desplegar una aplicación funcional que use inferencia con modelos preentrenados, sin entrenar un modelo propio.

Documento elaborado para explicar el problema, la elección del modelo, la arquitectura, el despliegue, las dificultades y los límites de la solución.

El problema

Documento elaborado para explicar el problema, la elección del modelo, la arquitectura, el despliegue, las dificultades y los límites de la solución.

1. El problema

BugTriage AI es una aplicación que utiliza inteligencia artificial para asistir en la clasificación inicial de reportes de errores de software (bugs). Está dirigida a desarrolladores, equipos de QA (Quality Assurance), testers y equipos de soporte que necesitan organizar los errores reportados y decidir a qué área conviene derivarlos.

El problema aparece cuando se reciben numerosos reportes escritos en lenguaje natural. Estos pueden ser breves, ambiguos, informales o contener términos técnicos. Clasificarlos manualmente requiere tiempo y puede generar diferencias de criterio entre las personas encargadas del proceso.

Problema concreto

La clasificación inicial de reportes puede ser lenta y subjetiva cuando los textos son breves, ambiguos o utilizan distintas formas de describir un mismo error.

Solución propuesta

BugTriage AI es una aplicación web que recibe un texto y propone una categoría entre cinco etiquetas: Funcional, UI/UX, Autenticación, Datos y Accesibilidad. La interfaz también presenta una puntuación del modelo, una estimación orientativa de nivel y categorías alternativas para ayudar a revisar la decisión.

Alcance y límites

El prototipo clasifica texto; no reproduce el error, no inspecciona código fuente, no crea tickets automáticamente y no determina de forma fiable la severidad real del incidente. La categoría sugerida es una primera orientación y requiere validación humana, sobre todo cuando el reporte es ambiguo o afecta varias áreas.

Usuarios destinatarios

Personas de QA, soporte técnico y desarrollo que necesitan ordenar reportes antes de asignarlos. En una futura integración, podría funcionar como etapa preliminar de un sistema de tickets, siempre con mecanismos de corrección y trazabilidad.

2. El modelo

Modelos evaluados

Para el proyecto se compararon dos modelos previamente entrenados que permiten realizar clasificación zero-shot. Esta técnica permite evaluar un texto frente a un conjunto de categorías propuestas sin entrenar un modelo específico con un dataset propio de reportes de bugs.

Modelo 1: facebook/bart-large-mnli

Model card: https://huggingface.co/facebook/bart-large-mnli

- Tarea: clasificación zero-shot mediante inferencia de lenguaje natural (Natural Language Inference, NLI).
- Entrenamiento: modelo BART-large ajustado con el conjunto MultiNLI.
- Idioma: el modelo y su entrenamiento se orientan principalmente al inglés; no está presentado como un modelo multilingüe especializado.
- Tamaño: aproximadamente 407 millones de parámetros. El archivo model.safetensors ronda los 1,63 GB; el repositorio completo puede ocupar más espacio porque contiene archivos adicionales y formatos alternativos.
- Uso en el proyecto: comparar la descripción del bug con las categorías candidatas y ordenar estas categorías según su puntuación.

Modelo 2: MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7

Model card: https://huggingface.co/MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7

- Tarea: clasificación zero-shot mediante NLI multilingüe.
- Entrenamiento: modelo multilingüe ajustado con datos de XNLI y conjuntos multilingües de inferencia de lenguaje natural.
- Idiomas: incluye español entre los idiomas compatibles; su entrenamiento se apoya en datos multilingües.
- Tamaño: aproximadamente 279 millones de parámetros.
- Uso en el proyecto: alternativa para evaluar si el soporte multilingüe ofrecía mejores resultados con los reportes utilizados.

Comparación experimental

La evaluación realizada en el notebook utilizó 12 casos de prueba que incluían textos cortos, informales, técnicos, en inglés y ambiguos.

Modelo	Accuracy reportada	Tiempo medio/caso
facebook/bart-large-mnli	50,00 % (6/12)	0,90 s
MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7	16,67 % (2/12)	4,15 s

Decisión

Se seleccionó facebook/bart-large-mnli porque obtuvo el mejor resultado observado y menor tiempo medio en este benchmark concreto. Aunque el segundo modelo es multilingüe, en estas pruebas rindió peor. Esta decisión no demuestra que BART sea superior en general: depende de las etiquetas, la redacción y la composición de los casos evaluados.

La aplicación usa clasificación zero-shot: en lugar de entrenar un clasificador propio con un conjunto etiquetado, se le entrega un texto y un conjunto de etiquetas candidatas. El modelo estima qué etiquetas se relacionan mejor con el contenido. En el proyecto se compararon dos modelos de inferencia preentrenados con los mismos 12 casos.

Cómo interpretar los resultados

La exactitud reportada corresponde al conjunto pequeño de casos utilizados en el notebook y no debe interpretarse como una garantía de rendimiento general. Las puntuaciones del modelo sirven para ordenar las etiquetas candidatas, pero no equivalen necesariamente a probabilidades calibradas de acierto.

¿Dónde falla el modelo y por qué?

La evaluación permitió observar que la clasificación zero-shot no siempre identifica la categoría esperada. El modelo fue desarrollado para tareas generales de inferencia de lenguaje natural, no específicamente para clasificar reportes de errores de software según las cinco categorías definidas en BugTriage AI.

Ejemplo 1: “El botón de guardar no funciona.”

- Categoría esperada: Funcional.
- Predicción de BART: UI/UX.
- Puntuación de la categoría predicha: aproximadamente 26,61 %.
- Resultado: clasificación incorrecta.

El modelo pudo asociar la palabra “botón” con la interfaz de usuario. Sin embargo, la expresión “no funciona” indica que podría existir un problema funcional. Sin más información, también podría tratarse de un problema de interacción o de diseño.

Este ejemplo muestra que una descripción breve puede no aportar suficiente contexto para distinguir entre un fallo funcional y uno de interfaz.

Ejemplo 2: “No anda.”

Una descripción tan corta no especifica qué componente falla, qué acción realizó la persona ni cuál fue el resultado esperado. Aunque una persona que conozca el contexto podría interpretar el reporte, el modelo recibe solamente el texto ingresado.

La falta de información dificulta la clasificación porque la categoría depende de datos que no están presentes en la descripción.

Ejemplo 3: “La pantalla de login se rompe cuando ingreso mi contraseña.”

Este caso permite observar que una descripción más contextualizada puede aportar información útil sobre el área afectada. En las pruebas del notebook, BART clasificó este reporte como Autenticación, con una puntuación aproximada de 30,75 %.

Aunque la categoría fue la esperada, la puntuación fue relativamente baja. Por lo tanto, acertar la categoría principal no significa necesariamente que el modelo esté seguro de su predicción.

Relación con los datos de entrenamiento

BART-large-MNLI fue ajustado para una tarea general de inferencia de lenguaje natural con MultiNLI, no para aprender directamente las categorías internas de BugTriage AI. Las etiquetas Funcional, UI/UX, Autenticación, Datos y Accesibilidad fueron definidas para este proyecto.

En consecuencia, el modelo compara relaciones semánticas entre el reporte y las categorías candidatas, pero no cuenta con una garantía de que comprenda los criterios particulares que un equipo de QA utilizaría para clasificar cada bug.

Además, el tamaño reducido del conjunto de evaluación —12 casos— limita las conclusiones que pueden extraerse. Una evaluación futura debería incluir más reportes reales, ejemplos de cada categoría y métricas como precisión, recall, F1 y matriz de confusión.

¿La aplicación entrena o infiere?

BugTriage AI realiza inferencia con un modelo previamente entrenado; no entrena ni ajusta los pesos del modelo durante el uso de la aplicación.

La aplicación carga el modelo mediante la función pipeline de la biblioteca Transformers:

```
clasificador = pipeline(
    "zero-shot-classification",
    model=MODELO,
    device=0 if torch.cuda.is_available() else -1
)
```

La variable MODELO identifica el modelo seleccionado. Transformers se encarga de cargar los componentes necesarios para realizar la inferencia.

Cuando el usuario ingresa un reporte, la aplicación lo procesa y obtiene puntuaciones para las categorías candidatas. Después utiliza esos resultados para presentar la categoría principal y las alternativas.

Inferencia local frente a inferencia remota

Durante la evaluación se probó el uso de un cliente remoto de inferencia, además de la vía con pipeline. La ruta que quedó funcionando en la aplicación usa el pipeline de Transformers dentro del entorno del Space. La prueba remota sirvió para identificar diferencias entre las interfaces de cliente, las credenciales y la disponibilidad de créditos del servicio.

Alternativa elegida: pipeline

La aplicación publicada utiliza pipeline de Transformers. Esta interfaz de alto nivel organiza el procesamiento necesario para ejecutar la tarea de clasificación, incluida la preparación de las entradas, la ejecución del modelo y la interpretación estructurada de sus resultados.

La aplicación carga el modelo en el entorno de Hugging Face Spaces y utiliza sus recursos para realizar las predicciones. Así, no depende de que cada solicitud se envíe a un proveedor externo de inferencia que requiera créditos disponibles.

Alternativa evaluada: InferenceClient

Durante las pruebas se encontraron problemas con esta alternativa. Primero, se utilizó una tarea no compatible con el modelo. Después, aunque se corrigió la llamada a la tarea de clasificación zero-shot, el proveedor devolvió un error HTTP 402 por falta de créditos disponibles.

Por este motivo, se mantuvo pipeline en la versión publicada.

3. Dónde corre cada cosa

La solución separa la interfaz web del proceso de inferencia, aunque en el despliegue actual la aplicación de Gradio y la carga del modelo están coordinadas desde el mismo proyecto.

Componentes

Componente	Responsabilidad	Dónde corre
Interfaz Gradio	Recibe el reporte, permite enviarlo y muestra categoría, score y alternativas.	Hugging Face Space
Pipeline Transformers	Prepara la inferencia zero-shot con las etiquetas candidatas.	Entorno Python del Space
Modelo BART MNLI	Calcula scores de compatibilidad entre el texto y las etiquetas.	Infraestructura de inferencia asignada al Space
Notebook de evaluación	Compara modelos con casos de prueba, calcula métricas y registra resultados.	Entorno local de desarrollo

Flujo de una consulta



Seguridad básica

Los tokens de acceso no deben escribirse en el código ni subirse al repositorio. Cuando una API necesita autenticación, el secreto debe configurarse como variable de entorno o secreto del servicio. Los mensajes de error deben evitar exponer credenciales o datos sensibles de los reportes.

4. El deploy

BugTriage AI está publicado como una aplicación Gradio en Hugging Face Spaces. El repositorio del Space contiene los archivos necesarios para construir y ejecutar la interfaz. La configuración de ejecución usa ZeroGPU a través de la configuración de Spaces del proyecto.

Enlace público

Aplicación publicada: https://huggingface.co/spaces/lagabyok/bugtriage-ai

Archivos centrales

- app.py: interfaz y lógica de clasificación.
- requirements.txt: dependencias Python.
- README.md: descripción e instrucciones del Space.
- notebook/: evaluación comparativa y archivos CSV con los resultados.

Proceso de publicación

El código se versiona con Git y se sincroniza con el repositorio del Space de Hugging Face. Los commits documentan la implementación, la configuración del Space, la actualización del README y la evaluación de modelos. El repositorio de GitHub del seminario es el destino académico para la entrega completa; conviene mantener separados los cambios de la aplicación desplegada y los materiales de evaluación/informe.

Prueba de funcionamiento

La app acepta texto en español y devuelve una categoría sugerida junto con información auxiliar. Una prueba con “El botón de guardar no funciona” mostró que la etiqueta de mayor score puede no coincidir con la interpretación humana más intuitiva. Este caso es útil para explicar por qué no debe tratarse la salida como una decisión definitiva.

Por qué Hugging Face Spaces

En este proyecto, Spaces permite publicar una interfaz accesible desde un navegador sin que cada usuario deba instalar Python, Transformers, PyTorch y el resto de las dependencias.

Comparación con otras alternativas

Alternativa	Ventajas	Desventajas para este proyecto
Hugging Face Spaces	Integración con Gradio y el ecosistema de modelos de IA; publicación sencilla.	Recursos, cuotas y disponibilidad sujetos al plan y a la configuración.
Hosting compartido	Puede ser económico para sitios web sencillos.	Habitualmente ofrece menos control sobre procesos Python y recursos de memoria necesarios para un modelo de este tamaño.
VPS	Mayor control sobre el sistema operativo, las dependencias, los procesos y la configuración.	Requiere administrar actualizaciones, seguridad, monitoreo, recursos y escalabilidad.
Render u otra plataforma PaaS	Simplifica la publicación de aplicaciones y servicios web.	El rendimiento y la viabilidad dependen de los recursos asignados, los límites del plan y el costo de ejecutar el modelo.
Arquitectura serverless	Puede escalar bajo demanda y evitar mantener una instancia dedicada de forma permanente.	Puede introducir arranques en frío, límites de memoria y tiempo de ejecución, además de complejidad adicional al cargar modelos grandes.

Se eligió Hugging Face Spaces porque responde a las necesidades de un prototipo académico: permite mostrar una aplicación funcional, integrar Gradio y ejecutar el modelo en un entorno administrado.

¿Por qué necesita un servidor o entorno de ejecución?

La interfaz debe recibir las solicitudes de los usuarios y ejecutar código Python para procesarlas. El modelo también necesita cargarse en memoria para realizar la inferencia. Por eso, la aplicación requiere un entorno de ejecución disponible cuando se procesan solicitudes.

Esto no significa que el plan gratuito garantice un servidor encendido permanentemente. Según la configuración, los recursos y las políticas vigentes, el Space puede tener limitaciones de uso, quedar inactivo o experimentar demoras al iniciarse.

Para un producto comercial, habría que evaluar la disponibilidad requerida, los recursos de CPU o GPU, la memoria, los tiempos de respuesta, los costos y la cantidad de solicitudes simultáneas.

5. Lo que se rompió

Durante el desarrollo surgieron problemas de configuración y disponibilidad que ayudaron a distinguir entre fallas de código, autenticación y límites del servicio.

Variable de entorno ausente

La primera prueba con el cliente remoto indicó que HF_TOKEN no estaba definido en el entorno. La lección fue verificar la configuración antes de inicializar el cliente y nunca imprimir ni guardar el token en archivos versionados. Se debe comprobar que el proceso pueda leer la variable de entorno sin imprimir el valor secreto y, finalmente, volver a ejecutar la prueba y comprobar que la autenticación esté configurada.

Error 401: Unauthorized

Después se obtuvo una respuesta 401 porque el cliente se inicializaba sin un token válido. Esto no indicaba necesariamente un problema en el modelo o en el texto de entrada; era un problema de credenciales/permisos.

Error 402: Payment Required

Luego apareció un error 402 por falta de créditos de inferencia disponibles. La conclusión es que una API remota puede depender de cuotas, permisos y condiciones de cuenta. Para un prototipo académico, hay que documentar esa dependencia y disponer de una ruta de ejecución alternativa.

Tarea incorrecta del cliente

La primera llamada probó text_classification, pero la tarea del modelo y el uso previsto requerían zero-shot classification. Se corrigió la llamada para usar zero_shot_classification con candidate_labels y multi_label=False. La interfaz del cliente debe coincidir con la tarea declarada por el modelo.

Mensajes técnicos registrados en el notebook

Falta de token:

```
ERROR: HF_TOKEN no está disponible.
Token cargado: False
```

Tarea incorrecta:

```
ValueError: Model 'facebook/bart-large-mnli' doesn't support task 'text-classification'. Supported tasks:
'zero-shot-classification', got: 'text-classification'
```

Créditos agotados:

```
HTTPStatusError: Client error '402 Payment Required' for url
'https://router.huggingface.co/hf-inference/models/facebook/bart-large-mnli'
You have no remaining credits. Purchase pre-paid credits to continue using Inference Providers. Alternatively,
subscribe to PRO to get monthly included credits.
```

Qué quedó funcionando

La aplicación publicada utiliza pipeline de Transformers dentro del entorno de Hugging Face Spaces, por lo que no depende de la solicitud remota que devolvió el error 402. La autenticación y los créditos siguen siendo relevantes si se vuelve a utilizar un proveedor externo.

6. Límites actuales y mejoras necesarias para producción

BugTriage AI es un prototipo académico funcional, pero no fue validado como un servicio de producción ni se realizaron pruebas de carga para demostrar que puede atender a 1.000 usuarios simultáneos.

¿Qué ocurriría con 1.000 usuarios?

El comportamiento dependería de cuántas solicitudes llegaran al mismo tiempo, los recursos asignados y la capacidad de procesamiento del entorno. Si la aplicación no dispone de suficiente memoria o capacidad de cómputo, podría aumentar la latencia, acumular solicitudes en espera, sufrir errores o dejar de responder temporalmente.

No es posible afirmar que la aplicación soporte esa cantidad de usuarios sin realizar pruebas de carga con escenarios representativos.

Límites identificados

- Calidad de clasificación: la exactitud observada fue del 50 % sobre un conjunto de 12 casos para el modelo seleccionado.
- Ambigüedad: los reportes breves pueden carecer del contexto necesario para elegir una categoría.
- Puntuación: el score del modelo no debe interpretarse como una probabilidad calibrada de que la clasificación sea correcta.
- Recursos: el modelo necesita memoria y capacidad de procesamiento para realizar inferencias.
- Disponibilidad: el servicio depende del entorno de ejecución, sus cuotas y sus condiciones de uso.
- Escalabilidad: no se midió el rendimiento con cientos o miles de solicitudes concurrentes.
- Privacidad: en una versión real sería necesario definir políticas de almacenamiento, retención y protección de los reportes ingresados.

Mejoras propuestas

1. Ampliar el conjunto de evaluación. Incorporar reportes reales anonimizados y etiquetados por personas con experiencia en QA.
2. Medir el rendimiento por categoría. Calcular precisión, recall, F1 y matriz de confusión para conocer qué tipos de bugs se clasifican peor.
3. Mejorar las categorías. Definir criterios y ejemplos claros para reducir la superposición entre Funcional y UI/UX, por ejemplo.
4. Agregar revisión humana. Permitir que el usuario confirme o corrija la categoría sugerida y utilizar esas correcciones para evaluar mejoras futuras.
5. Calibrar la confianza. Validar los scores antes de usarlos para tomar decisiones automáticas.
6. . Fortalecer el manejo de errores. Incorporar validaciones, mensajes claros, registros técnicos y recuperación ante fallos.
7. Mejorar la infraestructura. Evaluar recursos de CPU/GPU adecuados, colas de solicitudes, límites de uso y escalabilidad.
8. . Evaluar costos y disponibilidad. Si el proyecto se transforma en producto, elegir una infraestructura con capacidad y nivel de servicio acordes con la demanda real.

Estas mejoras permitirían pasar de una demostración académica a un sistema más robusto, medible y preparado para un uso profesional.

7. Vocabulario de programación aplicado a app.py

Los siguientes fragmentos corresponden al código de la aplicación publicada.

```
def clasificar_bug(texto):

    if not texto or not texto.strip():

        return "Ingresá un reporte de bug.", ""

    demo = gr.Interface(fn=clasificar_bug)

    demo.launch()
```

Elementos señalados:

- **Clase:** `gr.Interface` (molde/tipo de dato que define la interfaz en Gradio).
- **Instancia:** `demo` (objeto creado al llamar a `gr.Interface(...)`).
- **Método:** `strip()` (o también `launch()`, funciones que pertenecen y operan sobre un objeto).
- **Atributo:** `__name__` (o la propiedad interna del módulo que almacena su nombre de ejecución).
- **Parámetro:** `texto` en `def clasificar_bug(texto):` (o `fn` en `Interface`).
- **Argumento:** `clasificar_bug` pasado a `fn=` (o el valor real que recibe `texto` al ejecutarse).
- **Valor de retorno:** `"Ingresá un reporte de bug.", ""` (la tupla devuelta por la instrucción `return`).

8. Vocabulario técnico

Accuracy (exactitud). Proporción de predicciones correctas sobre el total de casos evaluados. Se calcula dividiendo la cantidad de predicciones correctas por la cantidad total de ejemplos evaluados.

API (interfaz de programación de aplicaciones). Conjunto de reglas y mecanismos que permite que dos programas o sistemas se comuniquen e intercambien información.

Benchmark (prueba comparativa). Conjunto de pruebas y procedimientos utilizados para comparar modelos o soluciones bajo condiciones similares.

Deploy (despliegue o publicación). Proceso de poner una aplicación en funcionamiento en un entorno donde otras personas pueden acceder a ella.

Gradio. Biblioteca de Python que permite construir interfaces web interactivas para modelos de machine learning y funciones de Python.

Hugging Face Spaces. Servicio de alojamiento que permite publicar y compartir demostraciones de aplicaciones de machine learning.

Inferencia. Uso de un modelo previamente entrenado para generar una salida a partir de una entrada nueva.

Modelo preentrenado. Modelo cuyos parámetros fueron aprendidos durante una etapa previa de entrenamiento. En este proyecto se utiliza un modelo existente, sin entrenarlo desde cero.

NLI (Natural Language Inference o inferencia de lenguaje natural). Tarea que evalúa la relación lógica entre dos enunciados, por ejemplo, si una hipótesis se desprende de una premisa. Algunos modelos NLI pueden utilizarse para realizar tareas de clasificación *zero-shot*.

Pipeline. Abstracción de la biblioteca Transformers que reúne las etapas necesarias para ejecutar una tarea, como el procesamiento de la entrada, la inferencia y el postprocesamiento de la salida.

Score (puntuación). Valor numérico que el modelo asigna a una etiqueta o predicción. No debe interpretarse automáticamente como una probabilidad calibrada ni como una medida de exactitud garantizada.

Token de acceso. Credencial utilizada para autenticar solicitudes a un servicio. Debe almacenarse de forma segura y no publicarse en repositorios ni archivos compartidos.

Triage (clasificación inicial de incidencias). Proceso de revisión preliminar que permite identificar, categorizar y derivar problemas según su tipo o prioridad.

Zero-shot classification (clasificación sin ejemplos específicos de entrenamiento). Técnica que permite clasificar un texto entre categorías candidatas sin entrenar previamente el modelo con un conjunto de datos específico de esa tarea. El modelo utiliza el conocimiento adquirido durante su entrenamiento previo.

API o cliente remoto. Mecanismo mediante el cual una aplicación solicita inferencias a un servicio externo. El procesamiento puede realizarse en un servidor remoto en lugar de ejecutarse localmente.

Nota técnica: las puntuaciones de un clasificador *zero-shot* permiten comparar las categorías propuestas, pero no garantizan que la predicción sea correcta. Para evaluar el rendimiento real de la aplicación, es necesario utilizar ejemplos etiquetados y métricas adecuadas al problema.

9. Conclusión

BugTriage AI demuestra un flujo de inferencia con un modelo preentrenado, una interfaz web y un despliegue público. La comparación inicial permitió elegir un modelo para el prototipo, pero también reveló límites de calidad, ambigüedad y dependencia de servicios.

Durante las pruebas observé que la clasificación puede presentar errores y que algunos reportes son ambiguos. También comprobé que un servicio externo de inferencia puede tener restricciones, como la falta de créditos.

El objetivo no es afirmar que la aplicación clasifica todos los errores correctamente, sino demostrar un prototipo funcional que pueda mejorar con más pruebas, métricas y revisión humana.

Enlaces

- Aplicación publicada: BugTriage AI en Hugging Face Spaces — https://huggingface.co/spaces/lagabyok/bugtriage-ai
- Repositorio del TP Integrador: GitHub — TP_Integrador — https://github.com/lagabyok/Gabriela-Coronel-seminario-de-actualizacion-2026/tree/main/TP_Integrador
- Notebook de evaluación: 01_evaluacion_modelos.ipynb — https://github.com/lagabyok/Gabriela-Coronel-seminario-de-actualizacion-2026/blob/main/TP_Integrador/notebook/01_evaluacion_modelos.ipynb

Incluye el código y los materiales de evaluación e informe del TP Integrador.

Declaración de uso de IA generativa

Se utilizaron herramientas de IA generativa como apoyo para organizar y revisar la documentación del proyecto y para repasar conceptos técnicos. La implementación, las pruebas, la selección de los modelos y la revisión de los resultados se contrastaron con el código y las salidas del proyecto.

La herramienta no reemplaza la responsabilidad sobre la veracidad, comprensión y defensa del trabajo.